

Neural ODE for nanoring array dynamics

This notebook trains a **Neural Ordinary Differential Equation (Neural ODE)** to model the dynamics of a nanoring array (AMR) in response to an external magnetic field (H).

The pipeline has two stages: a one-time preprocessing script and this training notebook.

Preprocessing Script (`examples/preprocess_nanorings_data.py`)

Raw AMR and H-field trajectories are first loaded and filtered so that the mean AMR signal lies within a narrow, centered range (e.g., -0.5 to 0.5, extended to the full signal range). Each trajectory is then split into a transient part and a main sequence, with AMR

From the transient part, the last few points of the AMR and H signals are extracted as contexts (c_y , c_H , which are initial conditions). These serve as trajectory-specific embeddings that condition the model; they are not strict initial conditions.

To help capture periodicity, sinusoidal time features are added: $\sin(2\pi t)$ and $\sin(4\pi t)$.

Finally, all training and validation data, along with contexts, are saved into a single file: `data/processed/nanorings_data.pkl`, which is loaded by the training notebook.

Training Notebook (This File)

The training notebook loads the processed dataset and trains a **DriftNet** model using the `train_with_validation` function from the `neural_dynamics` library, which handles training, validation, and early stopping.

For inference, a single validation trajectory is selected. The model is conditioned on its contexts, and the `rollout` function is used to produce a full prediction. The predicted trajectory is then compared to the ground truth.

The Neural ODE Model

The model learns the function f in the ODE

$$\frac{dy}{dt} = f(y(t), H(t), \sin(\omega t), \sin(2\omega t), c_H, c_y)$$

where:

- $y(t)$ is the AMR signal (the state being predicted),

- $H(t)$ is the external driving magnetic field,

- $\sin(\omega t), \sin(2\omega t)$ are periodic time features,

- c_H, c_y are the context vectors conditioning predictions on a given trajectory.

```
1 # --- 1. Setup: Imports and Configuration ---
2 import marimo as mo
3 import os
4 import torch
5 import numpy as np
6 from torch.utils.data import DataLoader, TensorDataset
7
8 # Add project root to path to allow local imports
9 import sys
10 project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
11 if project_root not in sys.path:
12     sys.path.append(project_root)
13
14 from config import DEVICE
15 from neural_dynamics.core.hyperparameters import NetworkArchitecture
16 from neural_dynamics.models.base import DriftNet
17 from neural_dynamics.training.base import train_with_validation
18 from neural_dynamics.training.evaluation import rollout_trajectory
19 from examples.utils.plotting import plot_nanoring_results
20
21 # --- 2. Load Config and Preprocessed Data ---
22 class NanoringsHyperparameters:
23     """Hyperparameters for the Nanorings digital twin experiment."""
24
25     # Training parameters
26     LEARNING_RATE: float = 3.4e-4
27     NUM_EPOCHS: int = 1024
28     BATCH_SIZE: int = 16
29     VALIDATION_SPLIT: float = 0.2
30     EARLY_STOPPING_PATIENCE: int = NUM_EPOCHS # in principle this could be r
31
32     # Data sampling parameters
33     PERCENTAGE_OF_FILES_TO_LOAD: float = 1.0
34     PERCENTAGE_OF_MAIN_SEQUENCE_TO_USE: float = 0.1
35     TARGET_CHANNEL: int = 0
36
37     # System dynamics parameters
38     SAMPLE_RATE: int = 3200 # Hz
39     H_FIELD_AMPLITUDE_UPDATE_RATE: int = 100 # Timesteps
40     TRANSIENT_LENGTH: int = 100
41     CONTEXT_POINTS: int = 5
42
43     # Model architecture
44     DRIFT_NET_ARCH: NetworkArchitecture = NetworkArchitecture(
45
```

```

46         # amr, h, sin(wt), sin(2wt), h_ctx, amr_ctx
47         input_size=1 + 1 + 2 + CONTEXT_POINTS + CONTEXT_POINTS,
48         hidden_sizes=[128, 128, 128],
49         output_size=1,
50     )
51
52
53 config = NanoringsHyperparameters()
54
55 output_dir = os.path.join(
56     "examples",
57     "output",
58     f"epochs-{config.NUM_EPOCHS}_batch_size-{config.BATCH_SIZE}_lr-{config.LR}"
59 )
60 os.makedirs(output_dir, exist_ok=True)
61 print(f"Output directory: {output_dir}")
62
63 data_path = os.path.join(project_root, "data/processed/nanorings_dataset.pt")
64 processed_data = torch.load(data_path)
65 print(f"Loaded preprocessed data from {data_path}")
66
67 # --- 3. Build Model ---
68 initial_drift_net = DriftNet(config.DRIFT_NET_ARCH).to(DEVICE)

```

```

1  # --- 4. Train the Model ---
2
3  # Move data to the target device once
4  for key in processed_data:
5      if isinstance(processed_data[key], torch.Tensor):
6          processed_data[key] = processed_data[key].to(DEVICE)
7
8  # Create DataLoaders that yield batches of full trajectories
9  train_dataset = TensorDataset(
10     processed_data["train_h_context"],
11     processed_data["train_amr_context"],
12     processed_data["train_h_main"],
13     processed_data["train_amr_main_norm"],
14     processed_data["train_sin_time_1"],
15     processed_data["train_sin_time_2"],
16 )
17 train_loader = DataLoader(train_dataset, batch_size=config.BATCH_SIZE, shuffle=True)
18
19 val_dataset = TensorDataset(
20     processed_data["val_h_context"],
21

```

```

22     processed_data["val_amr_context"],
23     processed_data["val_h_main"],
24     processed_data["val_amr_main_norm"],
25     processed_data["val_sin_time_1"],
26     processed_data["val_sin_time_2"],
27 )
28 val_loader = DataLoader(val_dataset, batch_size=config.BATCH_SIZE, shuffle=F
29
30 # Define the function that prepares a batch of trajectories for the model
31 def batch_preparation_fn(raw_batch, device):
32     (
33         batch_h_ctx, batch_amr_ctx, batch_h_main, batch_amr_target,
34         batch_sin_time_1, batch_sin_time_2
35     ) = raw_batch
36
37     # One-step teacher forcing targets (a.k.a. horizon-of-one training)
38     current_amr = batch_amr_target[:, :-1]
39     target_amr = batch_amr_target[:, 1:]
40     current_h = batch_h_main[:, :-1]
41     current_sin_1 = batch_sin_time_1[:, :-1]
42     current_sin_2 = batch_sin_time_2[:, :-1]
43
44     # Flatten B x T -> (B*T) x 1
45     current_amr_flat = current_amr.reshape(-1, 1)
46     target_amr_flat = target_amr.reshape(-1, 1)
47     current_h_flat = current_h.reshape(-1, 1)
48     current_sin_1_flat = current_sin_1.reshape(-1, 1)
49     current_sin_2_flat = current_sin_2.reshape(-1, 1)
50
51     # Expand contexts across time then flatten
52     B, T = current_amr.shape
53     # Expand contexts across time then flatten
54     B, T = current_amr.shape
55     h_ctx_expanded = batch_h_ctx.unsqueeze(1).expand(B, T, -1).reshape(-1, b
56     amr_ctx_expanded = batch_amr_ctx.unsqueeze(1).expand(B, T, -1).reshape(-
57
58 # Final network input
59 net_input = torch.cat([
60     current_amr_flat, current_h_flat, current_sin_1_flat, current_sin_2_
61     h_ctx_expanded, amr_ctx_expanded
62 ], dim=1)
63
64 # Final supervision target
65 dt = processed_data["dt"]
66 true_derivatives = (target_amr_flat - current_amr_flat) / dt
67

```

```

67
68     return net_input, true_derivatives
69
70 # Run the training loop
71 trained_drift_net, training_losses, validation_losses = train_with_validation(
72     model=initial_drift_net,
73     train_loader=train_loader,
74     val_loader=val_loader,
75     num_epochs=config.NUM_EPOCHS,
76     learning_rate=config.LEARNING_RATE,
77     early_stopping_patience=config.EARLY_STOPPING_PATIENCE,
78     device=DEVICE,
79     batch_preparation_fn=batch_preparation_fn,
80 )
81
82 # --- Save Losses for Debugging ---
83 np.savetxt(
84     os.path.join(output_dir, "losses.txt"),
85     np.column_stack([training_losses, validation_losses]),
86     header="Training Loss, Validation Loss",
87 )

```

```

1 # --- 5. Evaluate and Visualize ---
2 trained_drift_net.eval()
3
4 # Get one sample from the validation set for inference
5 traj_idx_to_plot = 0
6 val_min, val_max = processed_data["val_scalers"][traj_idx_to_plot]
7 h_min_val, h_max_val = processed_data["val_h_scalers"][traj_idx_to_plot]
8
9 with torch.no_grad():
10     # Get the necessary data for this trajectory from the processed file
11     y0_eval = (
12         processed_data["val_amr_main_norm"][traj_idx_to_plot, 0]
13         .unsqueeze(0)
14         .unsqueeze(0)
15     )
16     h_context_eval = processed_data["val_h_context"][traj_idx_to_plot].unsqueeze(0)
17     amr_context_eval = processed_data["val_amr_context"][traj_idx_to_plot].unsqueeze(0)
18
19     val_h_main_traj = processed_data["val_h_main"][traj_idx_to_plot]
20
21     # Define the example-specific drift function that captures context
22
23

```

```

24 def drift_func_eval(t, y):
25     idx = min(
26         int((t - processed_data["t_min"].item()) / processed_data["dt"]
27             val_h_main_traj.shape[0] - 1,
28     )
29     h_t = val_h_main_traj[idx].unsqueeze(0).unsqueeze(0)
30
31     t_norm = (
32         t
33         - processed_data["t_min"].item()
34     ) / (processed_data["t_max"].item() - processed_data["t_min"].item()
35     t_norm_tensor = torch.full((1, 1), t_norm, device=y.device, dtype=y
36     sin_t_1 = torch.sin(2 * torch.pi * t_norm_tensor)
37     sin_t_2 = torch.sin(4 * torch.pi * t_norm_tensor)
38
39     net_input = torch.cat(
40         [y, h_t, sin_t_1, sin_t_2, h_context_eval, amr_context_eval], d
41     )
42     return trained_drift_net(net_input)
43
44 # Use the generic rollout function to get the predicted trajectory
45 eval_num_steps_required = processed_data["val_amr_main_norm"].shape[1]
46 eval_final_time = (
47     processed_data["t_min"].item() + eval_num_steps_required * processe
48 )
49
50 _, pred_traj_norm = rollout_trajectory(
51     drift_function=drift_func_eval,
52     initial_state=y0_eval,
53     initial_time=processed_data["t_min"].item(),
54     final_time=eval_final_time,
55     timestep=processed_data["dt"],
56 )
57
58 predicted_sequence = pred_traj_norm.squeeze() * (val_max - val_min) + v
59 true_val_sequence = (
60     processed_data["val_amr_main_norm"][traj_idx_to_plot]
61     * (val_max - val_min)
62     + val_min
63 )
64
65 h_field_unnorm = (
66     val_h_main_traj * (h_max_val - h_min_val + 1e-8) + h_min_val
67 )
68
69

```

```

69
70     all_train_seqs = []
71     for i, (train_min, train_max) in enumerate(processed_data["train_scaler
72         train_seq_norm = processed_data["all_train_amr_main_norm"][i]
73         train_sequence = train_seq_norm * (train_max - train_min) + train_m
74         all_train_seqs.append(train_sequence.cpu().numpy())
75
76     np.savetxt(
77         os.path.join(output_dir, "predicted_trajectory.txt"),
78         predicted_sequence.cpu().numpy(),
79     )
80     np.savetxt(
81         os.path.join(output_dir, "true_trajectory.txt"),
82         true_val_sequence.cpu().numpy(),
83     )
84
85     plot_result = plot_nanoring_results(
86         training_losses=training_losses,
87         validation_losses=validation_losses,
88         time_axis=processed_data["time_grid"].cpu().numpy(),
89         true_sequence=true_val_sequence,
90         predicted_sequence=predicted_sequence,
91         h_field=h_field_unnorm,
92         all_train_sequences=np.array(all_train_seqs),
93         output_dir=output_dir,
94     )
95
96     # --- Adjust AMR axis tick labels to gradient from COLOURS[2] to COLOURS[3]
97     # Ensure COLOURS array is available; adjust import path as necessary
98     try:
99         from examples.parameters.nanorings import COLOURS
100     except Exception:
101         COLOURS = None
102
103     if COLOURS is not None and len(COLOURS) >= 4:
104         ax = plt.gca()
105         y_ticks = ax.get_yticks()
106         tick_labels = ax.get_yticklabels()
107
108         # Create a linear interpolation between the two colours
109         color_start = np.array(COLOURS[2]) / 255.0
110         color_end = np.array(COLOURS[3]) / 255.0
111         num_ticks = len(y_ticks)
112         interpolated_colors = [
113             tuple(color_start + (color_end - color_start) * i / max(num_ticks -
114                 for i in range(num_ticks)

```

```
115     ]
116
117     # Apply the interpolated colour to each tick label
118     for lbl, col in zip(tick_labels, interpolated_colors):
119         lbl.set_color(col)
120
121     # Gradient for the axis label (apply middle colour)
122     y_label = ax.get_ylabel()
123     if y_label:
124         middle_color = interpolated_colors[num_ticks // 2]
125         ax.set_ylabel(y_label, color=middle_color)
126
127     # Optional: set tick label font size and rotation if needed
128     ax.tick_params(axis="y", labelsz=10)
129
130     # Return the figure with modified tick colours
131     plt.gcf()
132 else:
133     # If COLOURS not available, just return the original figure
134     plot_result
```